

Introduccion a Java

Juan Ignacio Raggio

May 3, 2025

Contents

1	DOUBTS DUDAS poo	2
2	Que es Java?	2
3	Debugging	3
4	Clases abstractas	3
5	Clases built-in	3
6	Las variables de instancia (static)	3
7	Todas las clases heredan de la clase object	3
8	Desensamblador de los .class	3
9	Modificadores	4
10	Metodos estandar	4
11	Atencion eficiencia! - tipos built-in vs clases EFICIENCIA	5
12	Loops	5
12.1	Forma clasica	5
12.2	for-each loops	5
13	Arreglos	6
13.1	Clase arrays	6
14	Argumentos variables	6

15 Strings	6
15.1 StringBuilder	7
16 Excepciones	EXCEPTIONS 7
16.1 java.lang.Object	7
16.1.1 java.lang.Throwable	7
16.1.2 java.lang.Exception CHECKED_EXCEPTIONS . .	8
16.2 Excepciones verificadas -> extends Exception	8
16.3 Excepciones no-verificadas -> extends RuntimeException . .	8
16.4 Finally	8
17 Comparacion entre instancias	9
17.1 Nunca vamos a usar el == para comparar objetos	9
17.1.1 Este metodo equals() tiene que respetar la relacion de equivalencia y ser consistente (que no cambie el estado de la instancia)	9
17.2 Implementacion de equals:	9
17.3 Wrappers tienen compares hechos automaticamente (con el epsilom incluido)	9
17.4 hashCode()	9

1 DOUBTS DUDAS poo

2 Que es Java?

- No es OO puro
- No todo es un objeto
- Tipado estatico
- Tipos built-in
- Instrucciones de control
- Sintaxis similar a C
- No tiene herencia multiple
- Manejo de errores a travez de **excepciones**

3 Debugging

- jdb
- Remote Debugging

4 Clases abstractas

- Es un intermediario que sirve cuando dos clases tienen cosas en común pero no es herencia directa sino que comparten solo algunas cosas
- Es como un header

5 Clases built-in

- Existe el tipo de dato: `Integer`. A diferencia de `int`, es una clase y tiene métodos built-in
- Existen muchos más pero los vamos a ver más adelante

6 Las variables de instancia (static)

- Se inicializan en `null` automáticamente (en caso de ser clases)
- Se inicializan en `0` (en caso de ser datos simples)

7 Todas las clases heredan de la clase `Object`

- Esto quiere decir que no es necesario hacer un constructor explícito para una clase

8 Desensamblador de los `.class`

```
javap -v -p HelloWorld.class
```

Hace ingeniería inversa y me muestra el código “descompilado”

9 Modificadores

```
public abstract class Foo {

    public static int staticMethod() {...}; //-> Metodo de clase

    public final int finalMethod() {...};

    public abstract int abstractMethod(); //-> Metodo abstracto

    public synchronized int syncMethod() {...}; //-> Es para multithreading

    public native int nativeMethod(); //-> Esta implementado en C, C++ o assembly
}

public class Foo2 extends Foo {
    public abstract int otherAbstractMethod(); -> Esto me da un error porque no puedo de
        public int method() {...};
}
```

10 Metodos estandar

```
public class Date {
    private int day, month, year;

    @override
    public boolean equals(Object object) {
        return object instanceof Date aux && aux.day == this.day && aux.month == this.month
    }

    @override
    public String toString() {
        return "%d/%d/%d".formatted(day, month, year);
    }
}
```

11 Atencion eficiencia! - tipos built-in vs clasesEFICIENCIA

- Hay tipos built-in (en minuscula)
- Para cada tipo built-in hay una clase **wrapper** (en mayuscula) Es importante saber que para cuando usamos clases wrappers, se llama mucho a constructores por lo que es uno de los mayores problemas de eficiencia que tiene java
- Las promociones son igual que en C
- Las demociones dan error de compilacion

`Double avg = 0;` -> Esta mal porque el 0 es un tipo built-in, mientras que `Double`

12 Loops

- Las condiciones si o si tienen que ser expresiones booleanas, no cualquier valor distinto de 0 es true

12.1 Forma clasica

```
int[] v1 = new int[10];
int sum = 0; // Aca podriamos crear un Integer en vez de sum pero es ineficiente
for (int i = 0; i < v1.length; i++) {
    sum += v1[i];
}
```

12.2 for-each loops

- OJO: La meta-variable (en este caso i) no es un left-value, por lo que no puedo cambiar el contenido dentro de ese arreglo

```
double avg = 0;
for (int i : v1) {
    avg += i;
}
avg /= v1.length;
```

13 Arreglos

- Built-in y C-style array vs Wrapper y Java-style array

```
int v1[] = new int[10]; // Built-in C-style
Integer[] v2 = new Integer[10]; // Wrapper Java-style
v1[10]; -> Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
v2[1] + 5; -> Exception in thread "main" java.lang.NullPointerException

Object[] v = {"Este array", 0, 1.5, "no es homoganeo"};
```

13.1 Clase arrays

```
static int binarySearch(Object[] a, Object key);
static int binarySearch(long[] a, long key);
static void sort(int[] a);
...
```

14 Argumentos variables

- Lo tenemos que poner siempre último en el prototipo de la función

```
static int sum(int... values) {
    int res = 0;
    for (int n : values) {
        res += n;
    }
    return res;
}
```

15 Strings

- Los strings en Java son objetos inmutables, por lo que cuando queramos cambiar un string, creamos una nueva instancia ya que no podemos modificar la actual

```
String s1 = "Hello";
String s2 = "world";
String s3 = s1.concat(", ").concat(s2);
```

```
String s1 = "Hello";
String s2 = "world"; // Aca s1 y s2 no se vuelven a crear
String s3 = s1 + ", " + s2; // Hay que evitar la concatenacion en ciclos porque como lo
```

- Ese problema anteriormente mencionado surge muy comunmente cuando queremos leer de stdin

```
int ch;
String s; // Aca s no se inicializa, si o si hay que inicializar en java
while ((ch = System.in.read()) != -1 && ch != '\n') {
    s += ch;
}
s.toUpperCase(); // Aca no lo estamos guardando en nign lado, como la clase str

// La correccion seria:
s = s.toUpperCase(); // y ademas no usar la concatenacion para cada iteracion de
```

15.1 StringBuilder

- Lo que tiene string builder adentro es un arreglo de chars que lo va agrandando.
- Si queremos concatenar strings, si o si necesitamos usar StringBuilder, si no lo va a hacer de forma ineficiente

```
int ch;
StringBuilder s = new StringBuilder();
while ( (ch = System.in.read()) != -1 && ch != '\n' ) {
    s.append(ch);
}
String upperS = s.toString().toUpperCase();
```

16 Excepciones

EXCEPTIONS

Un evento que ocurre durante la ejecucion del programa que interrumpe el flujo normal

16.1 java.lang.Object

16.1.1 java.lang.Throwable

1. java.lang.Error

ERRORS

2. ...

16.1.2 java.lang.Exception

CHECKED_EXCEPTIONS

1. java.lang.CloneNotSupportedException

16.2 Excepciones verificadas -> extends Exception

- Catch or Specified -> Las excepciones que creamos nosotros a mano como clases, son verificadas y si o si tienen que mostrarse explícitamente
- Por ejemplo creo una clase InvalidDayException, si la quiero usar tengo que o atraparla o anunciarla

```
void method() throws InvalidDayException {
    if (..) {
        throw new InvalidDayException;
    }
}
// o
void method2() {
    try {
        ...
    } catch (InvalidDayException ex) {
        System.out.println(ex.getMessage);
    }
}
```

16.3 Excepciones no-verificadas -> extends RuntimeException

16.4 Finally

- La parte de bloque finally se ejecuta siempre.
- Salte o no la excepcion
- Se usa para cerrar recursos como archivos

17 Comparacion entre instancias

17.1 Nunca vamos a usar el == para comparar objetos

```
Persona p1 = new Persona("Jorge", 20_202_202);
Persona p2 = new Persona("Ivonne", 40_000_231);
p1 == p2; -> Va a ser siempre falso porque p1 y p2 son referencias
p1.equals(p2); -> Esto es lo que vamos a querer hacer pero necesitamos hacer un @Override
```

17.1.1 Este metodo equals() tiene que respetar la relacion de equivalencia y ser consistente (que no cambie el estado de la instancia)

17.2 Implementacion de equals:

```
@Override
public boolean equals(Object o) {
    // Primero nos fijamos si son comparables
    if (!(o instanceof MiClase aux)) // -> ocurre un downcasting y hace que aux pase a ser
        return false;
    if (propiedadUno != aux.propiedadUno) // -> Como se usa un != si o si propiedadUno es
        return false;
    return propiedadDos.equals(aux.propiedadDos); // -> Como se hace un equals, podria ser
}
```

17.3 Wrappers tienen compares hechos automaticamente (con el epsilon incluido)

Float.compare(float1, float2); -> Esto me hace la comparacion usando un epsilon automatico

17.4 hashCode()

```
@Override
public int hashCode() {
    return Objects.hash(propiedad1, propiedad2, propiedad3);
}
```

- Siempre que se sobrescriba el equals tiene que sobrescribirse el hashCode